

From ER to Relational Schema: SQL DDL

Databases and Software Architecture – Aalborg University

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.

From ER Diagrams to Relational Schemas

Last time we designed the library system as an ER diagram – entities, attributes, relationships, cardinality, participation – without writing a line of SQL. Today we turn that diagram into real tables: a systematic set of translation rules, then the SQL DDL to implement them with every constraint intact.

Agenda

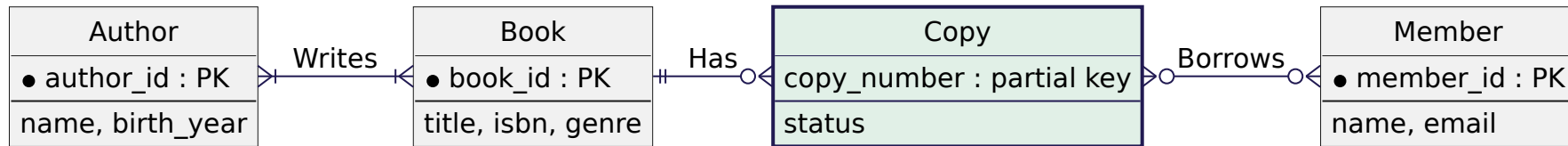
Eight parts, about 105 minutes: a recap, translation rules, SQL data types, a break, constraints and ALTER/DROP, cascade rules, live coding, and wrap-up.

- Recap: the library ER model from last lecture
- Translation rules: entity, 1:N, M:N, 1:1, weak entity → table
- SQL DDL part 1: data types and CREATE TABLE
- Break
- SQL DDL part 2: constraints, ALTER TABLE, DROP TABLE
- Cascade rules: ON DELETE CASCADE/RESTRICT/SET NULL/SET DEFAULT, and ON UPDATE
- Live coding: the complete library schema
- In-class exercise and wrap-up

Recap and Motivation

Recap: The Library ER Model

Last lecture we built this, piece by piece, ending on the full diagram: four entities, three relationships, all cardinalities and participation marked.



The library ER diagram from last lecture: Author, Book, Copy (weak), Member, and the Writes/Has/Borrows relationships

Borrows also carries `loan_date`, `due_date`, `return_date`, and derived `is_overdue` – omitted here, as before, to keep the diagram readable.

Recap: Entities, Attributes, Keys

- **Entity type** – Book, Copy, Author, Member; each has an **identifying attribute** (a chosen candidate key), underlined in the diagram
- **Attributes** come in four flavours – simple, composite, multi-valued (Book.genre), derived (Borrows.is_overdue)
- Copy is **weak** – no attribute of its own is globally unique; copy_number only disambiguates **within one book**

Recap: Relationships, Cardinality, Participation

- Writes (Book–Author) – M:N, **total** participation on both sides
- Has (Book–Copy) – 1:N, Copy's **identifying relationship**; Copy total, Book partial
- Borrows (Member–Copy) – M:N, both sides partial; carries relationship attributes

Every one of these facts is about to matter for how we write CREATE TABLE.

Today's Question: From Diagram to Tables

An ER diagram is DBMS-independent – it does not say how PostgreSQL should store any of this. Today answers that, with a **systematic** translation, not case-by-case guessing:

- Every entity type becomes **something** – usually a table
- Every relationship becomes **something** – a foreign key, or sometimes a whole new table
- The cardinality and participation we marked last time drive **exactly** which

Translation Rules: ER to Relational

The Translation Process: Overview

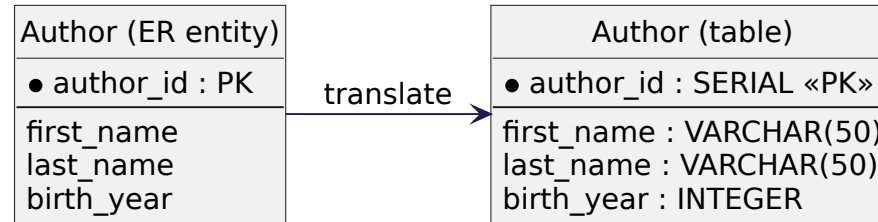
Five rules cover every ER diagram this course builds:

1. Entity → table
2. 1:N relationship → foreign key on the “many” side
3. M:N relationship → junction table with two foreign keys
4. 1:1 relationship → foreign key with UNIQUE
5. Weak entity → primary key including the owner’s primary key

We work through each with a diagram, then a real example from the library schema.

Rule 1: Entity → Table

Every entity type becomes a table. Every attribute becomes a column. The identifying attribute becomes the PRIMARY KEY.



An ER entity's attributes become a table's columns; the identifying attribute becomes PRIMARY KEY

The mechanical part stops there – **choosing SQL types and extra constraints** is SQL DDL Part 1 and Part 2's job, later in this lecture.

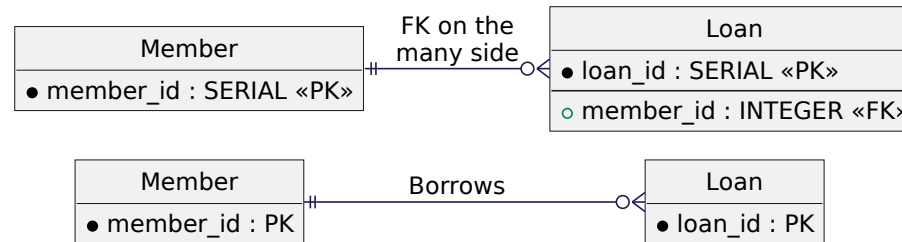
Rule 1 in Practice: Author

```
CREATE TABLE Author (  
    author_id    SERIAL PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    birth_year   INTEGER  
);
```

author_id was the underlined identifying attribute in the ER diagram – here it becomes SERIAL PRIMARY KEY. name was composite (first_name, last_name) – it becomes two plain columns, exactly as L2 already implied.

Rule 2: 1:N Relationship → Foreign Key

A 1:N relationship becomes a foreign key **on the “many” side**, referencing the “one” side’s primary key. No new table needed.



1:N: Loan (the “many” side) gets a foreign key column pointing at Member (the “one” side)

Rule of thumb: the crow’s foot end is where the foreign key column lives – exactly like `Student.programme_id` back in the first lecture.

Rule 2 in Practice: Member and Loan

One member has many loans; each loan belongs to exactly one member – 1:N.

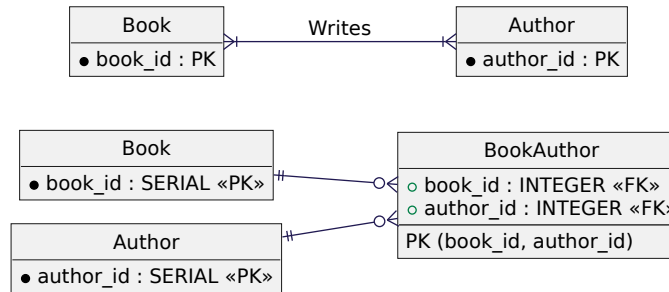
```
member_id  INTEGER NOT NULL REFERENCES Member(member_id) ON DELETE RESTRICT
```

This single line, inside Loan's definition, **is** the whole translation of that relationship. ON DELETE RESTRICT is the cascade-rule choice.

The Cascade Rules section, later in this lecture, explains why.

Rule 3: M:N Relationship → Junction Table

Neither side of an M:N relationship can hold a single foreign key – one column cannot point at “many” rows. The fix: a new table in the middle, with a foreign key to **each** side.



M:N: Writes becomes its own table, BookAuthor, with a foreign key to Book and a foreign key to Author

The junction table's primary key is usually the **pair** of foreign keys together – unless the relationship needs to repeat, which the Loan design, coming up in the live-coding walkthrough, revisits.

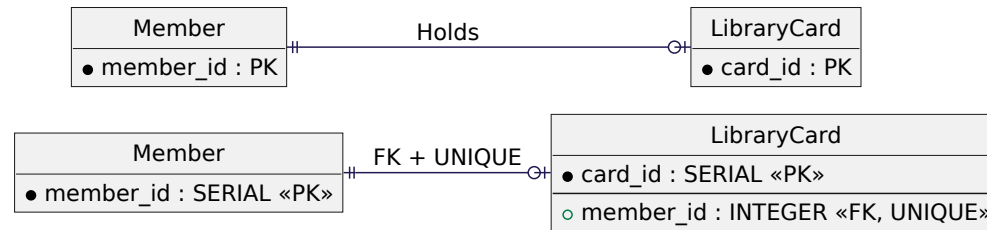
Rule 3 in Practice: Writes → BookAuthor

```
CREATE TABLE BookAuthor (  
    book_id    INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,  
    author_id  INTEGER NOT NULL REFERENCES Author(author_id) ON DELETE CASCADE,  
    PRIMARY KEY (book_id, author_id)  
);
```

The composite PRIMARY KEY (book_id, author_id) also does useful work as a constraint: it forbids recording the same book-author pair twice.

Rule 4: 1:1 Relationship → FK + UNIQUE

A 1:1 relationship is a 1:N foreign key with one extra constraint: UNIQUE on the foreign key column, so the “many” side collapses to “at most one.”



1:1: a foreign key exactly like Rule 2, plus UNIQUE so each Member can own at most one LibraryCard

Rule 4 in Practice: A Hypothetical LibraryCard

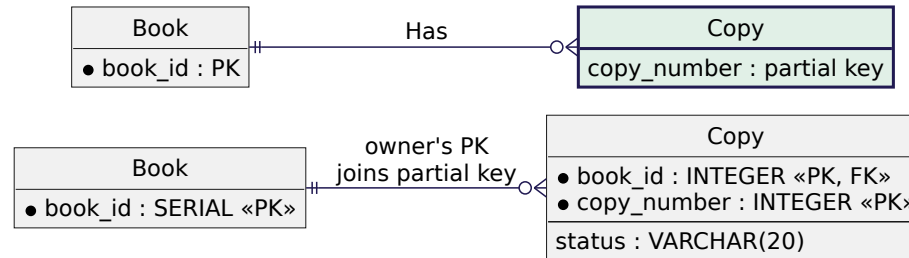
Hypothetical – not part of our shared schema, just illustrating the rule. Say each member could optionally hold one physical library card:

```
CREATE TABLE LibraryCard (  
    card_id    SERIAL PRIMARY KEY,  
    member_id  INTEGER UNIQUE REFERENCES Member(member_id)  
);
```

Drop UNIQUE and this is just Rule 2 again, 1:N – one member, many cards. UNIQUE is the **entire** difference between 1:N and 1:1 in SQL.

Rule 5: Weak Entity → Composite Primary Key

A weak entity's own attribute is not unique on its own. Its table's primary key must be the **pair**: the owner's primary key (as a foreign key) plus the partial key.



Weak entity: Copy's table gets a composite PRIMARY KEY combining Book's key (as a FK) with copy_number

Directly what L2 promised: “a weak entity's real key becomes a composite key: owner's key plus partial key.”

Rule 5 in Practice: Book and Copy

```
CREATE TABLE Copy (  
    book_id      INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,  
    copy_number  INTEGER NOT NULL,  
    status       VARCHAR(20) NOT NULL DEFAULT 'available',  
    PRIMARY KEY (book_id, copy_number)  
);
```

`book_id` does double duty – it is both a FOREIGN KEY (linking to the owner) **and** part of the PRIMARY KEY (giving `Copy` its identity). That double duty is the whole signature of a weak entity in SQL.

A Wrinkle: Multi-Valued Attributes Also Get Their Own Table

Not one of the five official rules, but the same instinct applies. Book.genre was multi-valued in the ER diagram – L2 already flagged it could not become a single column.

```
CREATE TABLE BookGenre (  
    book_id  INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,  
    genre    VARCHAR(30) NOT NULL,  
    PRIMARY KEY (book_id, genre)  
);
```

Shaped just like a junction table, but only one real foreign key – genre is a value, not another entity.

A Limitation: Total Participation Isn't Enforceable

Writes was **total** participation on both sides in the ER diagram – every book has at least one author, every author has at least one book. Plain SQL cannot enforce that minimum.

- FOREIGN KEY/NOT NULL express “at most one” cardinalities cleanly
- They cannot express “at least one” without triggers or application-level checks – both out of scope here
- Honest limitation, not an oversight: our BookAuthor table happily allows a book with zero rows in it

Translation Rules: Recap

ER construct	Relational translation
Entity	Table, one column per attribute
1:N relationship	FK on the “many” side
M:N relationship	Junction table, FK to each side
1:1 relationship	FK + UNIQUE
Weak entity	Composite PK: owner’s PK + partial key
Multi-valued attribute	Its own table, FK back to the owner

SQL DDL Part 1: Data Types and CREATE TABLE

SQL Data Types: Why They Matter

Every column needs a declared type. The type is what gives us **domain integrity**, back from the first lecture – values that do not match the type are rejected outright, before they ever reach a query.

- Too generic a type (everything as text) throws away that safety net
- Too narrow a type rejects legitimate data
- Picking well is part of translating the diagram, not an afterthought

We only cover the types this schema needs – PostgreSQL has many more (network addresses, geometric types, JSON, arrays, ...). Full reference: <https://www.postgresql.org/docs/current/datatype.html>.

Numeric Types

- INTEGER (or INT) – whole numbers, ± 2.1 billion – most primary and foreign keys (author_id, book_id, member_id)
- BIGINT – whole numbers, much larger range – for when INTEGER's ceiling might realistically matter (a high-volume log table, for instance; not needed in our library)
- SERIAL – INTEGER plus an auto-incrementing sequence – our surrogate primary keys, e.g. author_id SERIAL PRIMARY KEY
- NUMERIC(p, s) – exact fixed-point decimal – next slide

Text Types

- VARCHAR(n) – variable-length text, capped at n characters – when a sane maximum exists: title VARCHAR(200), email VARCHAR(100)
- TEXT – variable-length text, no cap – free-form content with no natural limit (this schema does not need it, but a book’s synopsis would be a good fit)
- PostgreSQL stores both the same way internally – the cap is a **data-quality** choice, not a performance one

Date, Time, and Boolean Types

- DATE – calendar date, no time-of-day – `join_date`, `loan_date`, `due_date`, `return_date`
- TIMESTAMP – date **and** time – when time-of-day matters (not needed here, but a `created_at` audit column would want it)
- BOOLEAN – true/false – simple flags. Not used in our schema either: `Copy.status` needs more than two states (“available,” “on loan,” “lost”), so it is text instead

NUMERIC(p, s): Exact Decimals

p = total digits, s = digits after the decimal point. Unlike floating-point types, NUMERIC never rounds unexpectedly – essential for money.

Bonus example – not part of our schema, but a natural fit if the library charged fines:

```
amount NUMERIC(6, 2) -- up to 9999.99
```

None of the seven library tables need a NUMERIC column, so it will not appear again after this slide.

Basic CREATE TABLE Syntax

```
CREATE TABLE table_name (  
    column1 TYPE [constraints],  
    column2 TYPE [constraints],  
    ...  
);
```

Each column: a name, a type, then zero or more constraints. We cover the constraints in depth shortly – for now, just the type.

CREATE TABLE in Practice: Author

```
CREATE TABLE Author (  
    author_id    SERIAL PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    birth_year   INTEGER  
);
```

Every type choice here has a reason: SERIAL for an auto-assigned surrogate key, VARCHAR(50) for names with a sane cap, plain INTEGER for a year (no need for DATE, since we only ever know the year).

Break



Stretch, grab water, ask now if any translation rule feels shaky. Next: constraints – the part of CREATE TABLE that turns “a table shaped like the diagram” into “a table that actually enforces it.”

SQL DDL Part 2:

Constraints, ALTER, DROP

Constraints: Why They Matter

A type alone only checks **shape** – VARCHAR does not stop an empty string, INTEGER does not stop a duplicate. Constraints enforce the **rules** the ER diagram promised: uniqueness, required values, valid ranges, references that must resolve.

Six kinds today: PRIMARY KEY, FOREIGN KEY ... REFERENCES, NOT NULL, UNIQUE, CHECK, DEFAULT.

Enforces **entity integrity**, from the first lecture: unique and never NULL. Can be one column, or several – a composite key, as every weak entity in this deck needs.

```
author_id SERIAL PRIMARY KEY           -- single-column
PRIMARY KEY (book_id, copy_number)     -- composite, written separately
```

The composite form is written as its own line, after all the columns it covers are declared.

Enforces **referential integrity**: a value must match an existing row in the referenced table, or be NULL.

```
member_id INTEGER NOT NULL REFERENCES Member(member_id) ON DELETE RESTRICT
```

Two ways to write it: inline after the column (as above), or as a separate FOREIGN KEY (...) REFERENCES ... (...) line – required whenever the key spans more than one column, as in Loan's reference to Copy.

- NOT NULL: this value is **required**, every row must supply one. `Book.title`, `Member.email`
- UNIQUE: no two rows may share this value, but NULL is still allowed (NULL does not count as duplicated).

`Book.isbn UNIQUE, Member.email ... UNIQUE`

- Combine both when a value is required **and** must be unique, as `Member.email` does

An arbitrary boolean condition every row must satisfy – constraints that do not fit PRIMARY KEY/FOREIGN KEY/NOT NULL/UNIQUE.

```
CHECK (return_date IS NULL OR return_date >= loan_date)
```

From Loan: a book cannot be returned before it was borrowed. Either the loan is still open (return_date IS NULL), or the dates are in the right order.

Supplies a value automatically when INSERT omits the column.

```
join_date  DATE NOT NULL DEFAULT CURRENT_DATE  
status     VARCHAR(20) NOT NULL DEFAULT 'available'
```

CURRENT_DATE is evaluated **at insert time**, not when the table was created – every new member's join_date is today's date, whenever “today” happens to be.

Constraints Recap: Loan

Five of six constraint kinds, in one table (no UNIQUE needed here):

```
CREATE TABLE Loan (  
    loan_id        SERIAL PRIMARY KEY,  
    member_id      INTEGER NOT NULL REFERENCES Member(member_id) ON DELETE RESTRICT,  
    book_id        INTEGER NOT NULL,  
    copy_number    INTEGER NOT NULL,  
    loan_date      DATE NOT NULL DEFAULT CURRENT_DATE,  
    due_date       DATE NOT NULL,  
    return_date    DATE,  
    FOREIGN KEY (book_id, copy_number)  
        REFERENCES Copy(book_id, copy_number) ON DELETE RESTRICT,  
    CHECK (return_date IS NULL OR return_date >= loan_date)  
);
```

PRIMARY KEY, two FOREIGN KEYS (one inline, one composite), NOT NULL, DEFAULT, and CHECK – all in one table. The live-coding walkthrough explains why Loan looks exactly like this.

ALTER TABLE: Adding and Dropping Columns

Schemas change after creation. ALTER TABLE edits an existing table in place.

```
ALTER TABLE Book ADD COLUMN language VARCHAR(20);
```

```
ALTER TABLE Member DROP COLUMN join_date;
```

Adding a column defaults every existing row's value to NULL (or a stated DEFAULT); dropping one deletes that column's data permanently, for every row.

ALTER TABLE: Adding Constraints

```
ALTER TABLE Book  
  ADD CONSTRAINT book_isbn_unique UNIQUE (isbn);
```

Useful for constraints you did not anticipate at CREATE TABLE time – hypothetically, if `Book.isbn` had not already been declared UNIQUE back then, this is how you would add it after the fact. It can **fail** immediately if existing data already violates it, e.g. two books already sharing an `isbn`.

```
DROP TABLE IF EXISTS BookGenre;
```

Deletes the table and all its data, irreversibly. IF EXISTS avoids an error if the table is already gone – handy in scripts you might re-run. (A separate DROP TABLE ... CASCADE also exists, for dropping a table that other tables still reference – not to be confused with ON DELETE CASCADE, covered next.)

Cascade Rules

Deleting Data: What Should Happen?

`DELETE FROM Book WHERE book_id = 3;` – but BookGenre, Copy, and BookAuthor rows still point at book 3. What does PostgreSQL do with them? `ON DELETE` says, per foreign key, one of four things.

- `CASCADE` – delete the referencing rows too
- `RESTRICT` – refuse the delete outright, while referencing rows exist
- `SET NULL` – keep the referencing rows, blank out the foreign key column
- `SET DEFAULT` – keep the referencing rows, reset the foreign key column to its `DEFAULT` value

The referencing rows **cannot meaningfully outlive** the row they point at – so remove them together.

```
book_id INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE
```

Used by BookGenre, Copy, and BookAuthor toward Book: a genre entry, a physical copy, or an authorship record all belong entirely to one book. Delete the book, and none of them mean anything any more.

The referencing rows represent **history** that should never disappear silently – force an explicit decision instead.

```
member_id INTEGER NOT NULL REFERENCES Member(member_id) ON DELETE RESTRICT
```

Used by Loan toward both Member and Copy: deleting a member or a copy must not quietly erase loan history. PostgreSQL refuses the delete while loans reference them – the librarian must handle those loans first.

Not used anywhere in this schema – but worth knowing for when a foreign key is **optional** and the referenced row's identity is not essential to keep.

Conceptual example: if `Loan` had an optional `handled_by_staff_id` column, `SET NULL` would fit there – deleting that staff member should not delete the loan record, just blank out who processed it.

```
handled_by_staff_id INTEGER REFERENCES Staff(staff_id) ON DELETE SET NULL --  
hypothetical
```

Requires the column to allow `NULL` in the first place – `SET NULL` cannot apply to a `NOT NULL` foreign key.

Like SET NULL, but resets the foreign key to its column's DEFAULT value instead of to NULL – requires that column to actually have a DEFAULT declared, typically a sentinel “placeholder” row rather than leaving the fact unknown.

```
handled_by_staff_id INTEGER REFERENCES Staff(staff_id)
    DEFAULT 0 ON DELETE SET DEFAULT -- hypothetical: staff_id 0 = "Unassigned"
```

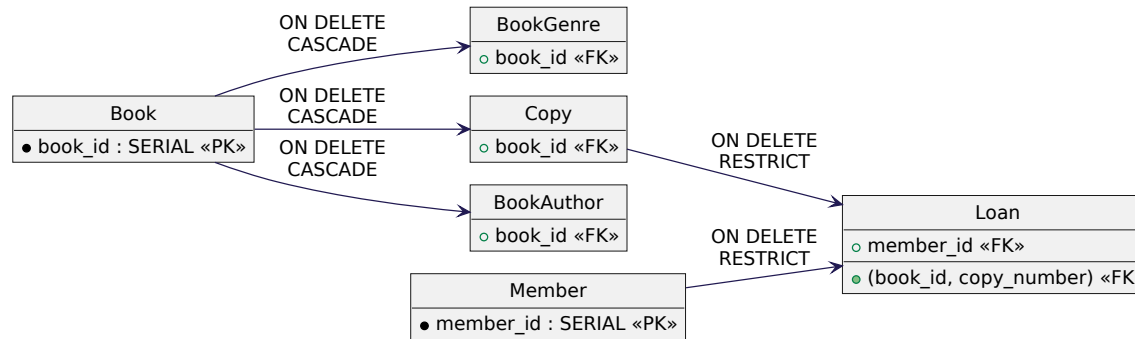
Not used anywhere in this schema – SET NULL already covers our one optional-FK case, and inventing a sentinel “unknown staff member” row just to avoid NULL would be unnecessary complexity here.

Every foreign key can independently choose a rule for ON UPDATE too, using the **same** options – but answering a different question: what happens to referencing rows if the **referenced row's key value itself changes**, not if the row is deleted.

```
book_id INTEGER NOT NULL REFERENCES Book(book_id)  
ON DELETE CASCADE ON UPDATE CASCADE
```

ON UPDATE CASCADE here means: if book_id 3 were ever renumbered, every referencing row would follow it automatically. In this schema, every primary key is a SERIAL surrogate value, never intentionally changed after INSERT – so ON UPDATE's choice rarely matters **here**. It matters more for schemas keyed on a natural value that can legitimately be renamed, e.g. a country_code primary key.

Cascade Rules in the Library Schema



CASCADE from Book to its three dependants; RESTRICT from Member and Copy toward Loan

Read it as a policy statement: delete a book, and everything that only exists **because of** that book goes with it. Delete a member or a copy, and PostgreSQL stops you until the loan history is dealt with explicitly.

Cascade Rules: Recap

Option	Effect	Used for
CASCADE	Delete referencing rows too	BookGenre, Copy, BookAuthor → Book
RESTRICT	Refuse the delete	Loan → Member, Loan → Copy
SET NULL	Blank the FK column	Not used here – optional FKs only, e.g. <code>handled_by_staff_id</code>
SET DEFAULT	Reset the FK column to its DEFAULT	Not used here – would need a sentinel placeholder row

Every option above also exists as `ON UPDATE` – same choices, triggered by the referenced key **changing** instead of being **deleted**. Not shown separately here since every key in this schema is a `SERIAL` surrogate.

Live Coding: The Complete Library Schema

We now build all seven tables together, in the order their foreign keys demand – a table cannot reference one that does not exist yet. Author, Book, and Member first, since nothing else depends on anything but them; then the tables that reference them.

Step 1: CREATE TABLE Author

```
CREATE TABLE Author (  
    author_id    SERIAL PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    birth_year   INTEGER  
);
```

No foreign keys – Author can be created first, in any order relative to Book and Member.

Step 2: CREATE TABLE Book

```
CREATE TABLE Book (  
    book_id          SERIAL PRIMARY KEY,  
    title            VARCHAR(200) NOT NULL,  
    isbn             VARCHAR(13) UNIQUE,  
    publication_year INTEGER  
);
```

isbn is UNIQUE but not NOT NULL – older or informally catalogued books may not have one on file. PostgreSQL actually ships a dedicated data type for ISBNs¹, but we keep things simple with VARCHAR(13) here.

¹PostgreSQL has a dedicated isbn data type (via the isn extension), but for simplicity this course sticks with plain VARCHAR(13). See <https://www.postgresql.org/docs/current/isn.html>.

Step 3: CREATE TABLE Member

```
CREATE TABLE Member (  
    member_id    SERIAL PRIMARY KEY,  
    first_name   VARCHAR(50) NOT NULL,  
    last_name    VARCHAR(50) NOT NULL,  
    email        VARCHAR(100) UNIQUE NOT NULL,  
    join_date    DATE NOT NULL DEFAULT CURRENT_DATE  
);
```

Three independent tables done. Everything from here on references at least one of them.

Step 4: CREATE TABLE BookGenre

-- Book.genre was multi-valued in the ER model -> its own table

```
CREATE TABLE BookGenre (  
    book_id  INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,  
    genre    VARCHAR(30) NOT NULL,  
    PRIMARY KEY (book_id, genre)  
);
```

First table that needs Book to already exist. ON DELETE CASCADE, exactly as decided in the Cascade Rules section a moment ago – a genre entry cannot mean anything without its book.

Step 5: CREATE TABLE Copy

```
-- Copy is a weak entity identified by (book_id, copy_number)
CREATE TABLE Copy (
    book_id      INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,
    copy_number  INTEGER NOT NULL,
    status       VARCHAR(20) NOT NULL DEFAULT 'available',
    PRIMARY KEY (book_id, copy_number)
);
```

Rule 5 in action – composite PRIMARY KEY (book_id, copy_number), exactly as diagrammed earlier in this lecture.

Step 6: CREATE TABLE BookAuthor

```
-- Writes (M:N, Book<->Author) -> junction table
CREATE TABLE BookAuthor (
    book_id    INTEGER NOT NULL REFERENCES Book(book_id) ON DELETE CASCADE,
    author_id  INTEGER NOT NULL REFERENCES Author(author_id) ON DELETE CASCADE,
    PRIMARY KEY (book_id, author_id)
);
```

Rule 3 in action – the junction table needed for `writes`, referencing both `Book` and `Author`.

Step 7: CREATE TABLE Loan

-- Borrows (M:N, Member<->Copy, WITH relationship attributes, and a member can
-- borrow the same copy again later) -> junction table needs a surrogate key,
-- not just the two FKs as PK, because the pair can legitimately repeat

```
CREATE TABLE Loan (  
    loan_id          SERIAL PRIMARY KEY,  
    member_id        INTEGER NOT NULL REFERENCES Member(member_id) ON DELETE RESTRICT,  
    book_id          INTEGER NOT NULL,  
    copy_number      INTEGER NOT NULL,  
    loan_date        DATE NOT NULL DEFAULT CURRENT_DATE,  
    due_date         DATE NOT NULL,  
    return_date      DATE,  
    FOREIGN KEY (book_id, copy_number)  
        REFERENCES Copy(book_id, copy_number) ON DELETE RESTRICT,  
    CHECK (return_date IS NULL OR return_date >= loan_date)  
);
```

All seven tables now exist. Two things about Loan are worth a slide of their own.

Why Loan Needs a Surrogate Key

Borrows is M:N with relationship attributes – Rule 3 says “junction table, PK = the two foreign keys.” Why does Loan break that pattern with its own `loan_id`?

- (`member_id`, `book_id`, `copy_number`) looks like the natural composite key
- But a member can borrow the **same physical copy** again later, at a different time – the pair would repeat
- A PRIMARY KEY can never repeat, so the composite pair cannot be the key here
- `loan_id` SERIAL sidesteps the problem: every loan event gets its own identity, regardless of repeats

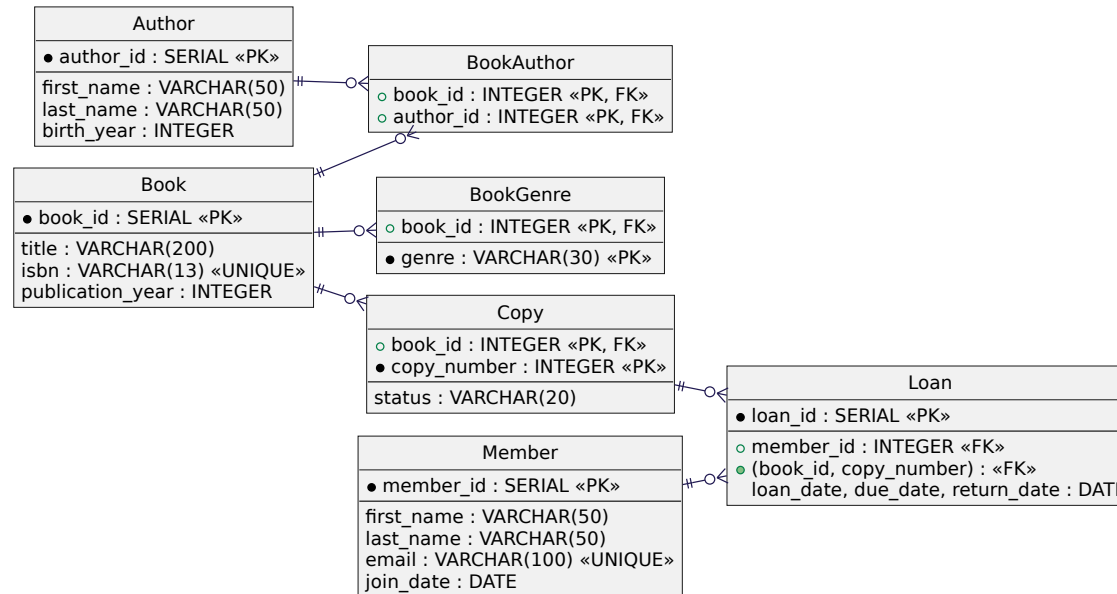
Loan's Composite Foreign Key

```
FOREIGN KEY (book_id, copy_number)  
  REFERENCES Copy(book_id, copy_number) ON DELETE RESTRICT
```

Directly caused by `Copy` having a composite primary key. A foreign key must match the **shape** of the primary key it references – since `Copy` needs both columns to identify one row, so does anything pointing at it. Same principle as L2's “a weak entity's real key is composite: owner's key + partial key,” now visible in a foreign key instead of a primary key.

The Complete Library Schema

All seven tables, primary keys and foreign keys marked, crow's-foot cardinalities carried over from the ER diagram.



The complete library relational schema: Author, Book, Member, BookGenre, Copy, BookAuthor, Loan

Computing `is_overdue` at Query Time

L2's derived attribute, `Borrows.is_overdue`, is deliberately **not** a column – computed instead, whenever we need it:

```
SELECT *, (return_date IS NULL AND due_date < CURRENT_DATE) AS is_overdue
FROM Loan;
```

This is not just a simplification – PostgreSQL's `GENERATED ALWAYS AS (...) STORED` columns require an **immutable** expression, and `CURRENT_DATE` is not immutable (it changes every day without an `UPDATE`). A generated `is_overdue` column genuinely could not work here, even if we wanted one.

Wrap-Up

Introduction to the In-Class Exercise

Translate today's five rules onto the library ER diagram yourselves: write the `CREATE TABLE` statements, execute them in PostgreSQL, then insert test data.

- Full instructions, and the dataset to test against, are on the exercise sheet
- Work table by table, in foreign-key order, exactly as the live coding did
- Compare your DDL with a neighbouring group – did you choose the same cascade rules?

Java Connection: SQL Types vs. Java Types

SQL types do not map one-to-one onto Java types – worth knowing now, ahead of JDBC later in the course.

SQL type	Java type
INTEGER / SERIAL	int / Integer
BIGINT	long / Long
VARCHAR(n) / TEXT	String
DATE	java.time.LocalDate
TIMESTAMP	java.time.LocalDateTime
BOOLEAN	boolean / Boolean
NUMERIC(p, s)	java.math.BigDecimal

Today: five translation rules from ER diagram to relational tables, SQL data types, CREATE TABLE, constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT), ALTER/DROP TABLE, ON DELETE/ON UPDATE cascade rules, and the complete library schema.